CHAPTER 02

Methods & Collections

C# Programming

Built-in methods • User-defined methods
Arrays • Lists • Dictionaries • LINQ

```
static int Add(int a, int b)
{
    return a + b;
}

var total = Add(10, 20);
Console.WriteLine(total);
```

LEARN • UNDERSTAND • PRACTISE

What we'll learn

Four building blocks for organizing code and working with collections.

01 Methods

Built-in methods, user-defined methods, parameters and return values.

02 Built-in tools

Math, String, Console, conversion, and date/time methods.

03 Collections

Arrays, List<T>, and Dictionary<TKey, TValue>.

04 LINQ & assignment

Use Where, Select, and OrderBy; analyse meter readings.

What is a method?

A method is a named block of code that performs a task when it is called.

Define once

Write the instructions in one place. A method can be reused whenever the task is needed.

Call when needed

Use the method name followed by parentheses to run its instructions.

Keep code organized

Break a large program into smaller, easier-to-understand tasks.

BUILT-IN METHODS **Provided by .NET**

USER-DEFINED **Created by the programmer**

Parameters, arguments & returns

Methods can receive data, process it, and send a result back.

C#

```
01  static int Add(int first, int second)
02  {
03      return first + second;
04  }
05
06  int total = Add(10, 20);
07  Console.WriteLine(total);
```

Here: first and second are parameters; 10 and 20 are arguments; 30 is returned.

Parameters

Named variables in the method declaration.

Arguments

Actual values passed when calling the method.

Return value

The result sent back using return.

Built-in methods in C#

Predefined methods are provided by classes in the .NET Class Library.

Methods are organized into classes

For example, Math contains numeric methods, String contains text methods, and Console handles terminal input and output.

- System.Math
- System.String
- System.Console

COMMON GROUPS

01 Mathematical calculations

02 String manipulation

03 Console input / output

04 Data type conversion

05 Date and time

Mathematical methods: Math

Use the Math class for common numeric calculations.

C#

```
01  double absolute = Math.Abs(-12);
02  double high = Math.Max(10, 25);
03  double low = Math.Min(10, 25);
04  double rounded = Math.Round(3.6);
05  double root = Math.Sqrt(25);
06  double power = Math.Pow(2, 3);
```

WHAT EACH METHOD DOES

Abs	Absolute value
Max / Min	Highest / lowest
Round	Nearest whole number
Sqrt	Square root
Pow	Power / exponent

String methods: clean and change text

String methods help combine, clean, and transform text.

C#

```
string text = " C# Programming ";
string clean = text.Trim();
string upper = clean.ToUpper();
string lower = clean.ToLower();
string changed =
clean.Replace("C#", "CSharp");
```

METHODS TO KNOW

Concat()	Combines strings
Join()	Joins values with a separator
Trim()	Removes outer whitespace
ToUpper()	Converts to uppercase
ToLower()	Converts to lowercase
Replace()	Replaces matching text

String methods: check text

Some string methods search for content and return a Boolean result.

C#

```
01 string text = "C# Programming";
02 bool hasCSharp = text.Contains("C#");
03 bool startsWithC = text.StartsWith("C");
04 bool endsWith = text.EndsWith("ing");
05 bool isBlank =string.IsNullOrEmpty(text);
```

CHECKING METHODS

`Contains(value)`
Checks whether text exists

`StartsWith()`
Checks the beginning

`EndsWith()`
Checks the ending

`IsNullOrEmpty()`
Checks null or empty text

Console input and output

Use Console methods to communicate through the terminal.

FOUR METHODS

WriteLine()
Prints and moves to next line.

Write()
Prints on the same line.

ReadLine()
Reads text until Enter.

ReadKey()
Waits for one key press.

C#

```
01 Console.Write("Enter your name: ");
02 string name = Console.ReadLine();
03 Console.WriteLine($"Hello, {name}!");
```

SAMPLE TERMINAL

```
Enter your name: Asha
Hello, Asha!
```

Converting data types

Convert text input into numeric values when calculations are needed.

C#

```
01  string textNumber = "123";
02  int number = int.Parse(textNumber);
03  double price = double.Parse("19.95");
04
05  if (int.TryParse("abc", out int value))
06  {
07      Console.WriteLine(value);
08  }
```

TryParse returns true when conversion succeeds; invalid text does not throw a parse error.

COMMON OPTIONS

`Convert.ToInt32()`
Converts to an integer

`Convert.ToString()`
Converts to text

`int.Parse()`
Parses integer text

`double.Parse()`
Parses decimal text

`int.TryParse()`
Safely checks conversion

Date and time methods

DateTime provides values and methods for working with dates.

C#

```
DateTime current = DateTime.Now;
DateTime today = DateTime.Today;
DateTime nextWeek = current.AddDays(7);
DateTime nextMonth = current.AddMonths(1);
DateTime nextYear = current.AddYears(1);
```

DATE / TIME MEMBERS

`Now`	Current date and local time
`Today`	Today at midnight
`AddDays(x)`	Moves by days
`AddMonths(x)`	Moves by months
`AddYears(x)`	Moves by years

User-defined methods (UDFs)

A user-defined method is written by the programmer to perform a specific task.

WHY CREATE METHODS?

- Reuse instructions
- Break problems into smaller tasks
- Make code easier to read
- Simplify testing and changes

C#

```
01  static void FunctionName(int value)
02  {
03      // Code to execute
04      Console.WriteLine(value);
05  }
06
07  FunctionName(10);
```

A method with parameters: void

Use parameters to send values into a method; void means it returns no value.

C#

```
01 static void AddNumbers(int first, int second)
02  {
03      int result = first + second;
04      Console.WriteLine($"Sum: {result}");
05  }
06
07  AddNumbers(10, 20);
```

IN THIS EXAMPLE

`first, second`

Parameters declared in the method.

`10, 20`

Arguments supplied when calling it.

OUTPUT: 30

Methods with a return value

A return type tells C# what kind of value the method sends back.

C#

```
01  static int FunctionName(int value)
02  {
03      int result = value * 2;
04      return result;
05  }
06
07  int answer = FunctionName(5);
```

REMEMBER

int

The method returns an integer.

return result;

Sends the calculated value back to the caller.

Complete example: return a greeting

The calling code stores the returned string and then displays it.

C#

```
01 static string GreetUser(string name, int age)
02  {
03      return $"Hello {name}! " +
04             $"You are {age} years old.";
05  }
06
07  string message = GreetUser("Alice", 28);
08 Console.WriteLine(message);
```

OUTPUT

```
Hello Alice!
You are 28 years old.
```

Trace it

The method receives Alice and 28, builds a string, and returns it.

Arrays: fixed-size collections

An array stores multiple values of the same data type in a fixed-size sequence.

C#

```
01 int[] luckyNumbers = { 7, 13, 21, 42 };
02
03 Console.WriteLine(luckyNumbers[0]);
04 Console.WriteLine(luckyNumbers[2]);
```

ARRAY INDEXES START AT 0



luckyNumbers[0] is 7.
The last index is Length - 1.

Multidimensional arrays

A 2D array arranges values in rows and columns, like a grid.

C#

```
int[,] grid =
{
    { 1, 2, 3 },
    { 4, 5, 6 }
};

Console.WriteLine(grid[1, 2]);
```

2 ROWS × 3 COLUMNS

1	2	3
4	5	6

grid[1, 2] gives 6

List<T>: a collection that can grow

A List<T> stores values of one type and can grow or shrink as items are added or removed.

C#

```
01  using System.Collections.Generic;
02
03  List<int> readings = new List<int>
04  { 120, 98, 135 };
05
06  readings.Add(110);
07  readings.Remove(98);
08  Console.WriteLine(readings[0]);
09  Console.WriteLine(readings.Count);
```

USEFUL LIST MEMBERS

`Add(value)`	Adds an item
`Remove(value)`	Removes a matching item
`Count`	Number of items
`[index]`	Access by position

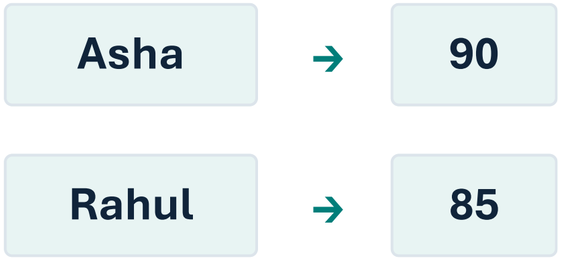
Dictionary<TKey, TValue>

A dictionary stores key–value pairs; each key must be unique.

C#

```
01  using System.Collections.Generic;
02
03  Dictionary<string, int> marks =
04      new Dictionary<string, int>();
05
06  marks["Asha"] = 90;
07  marks["Rahul"] = 85;
08  Console.WriteLine(marks["Asha"]);
09  Console.WriteLine(marks.ContainsKey("Rahul"));
```

KEY → VALUE



Access a value using its key. Use ContainsKey() before lookup when needed.

A first look at LINQ

LINQ helps query and transform data in collections using readable expressions.

Language Integrated Query

LINQ provides a consistent way to filter, select, and sort data from a collection.

Where()	Filters matching items
Select()	Transforms each item
OrderBy()	Sorts the sequence

C#

```
using System.Linq;

var high = readings.Where(r => r > 100);

var doubled = readings.Select(r => r * 2);
var sorted = readings.OrderBy(r => r);
```

LINQ example: Where, Select & OrderBy

Start with one collection, then apply a query operation to it.

C#

```
01  List<int> readings = new List<int>
02  { 120, 98, 135, 110, 90 };
03
04  var high = readings.Where(r => r > 110);
05  var doubled = readings.Select(r => r * 2);
06  var sorted = readings.OrderBy(r => r);
```

INPUT READINGS

120, 98, 135, 110, 90

Where > 110
120, 135

Select × 2
240, 196, 270, 220, 180

OrderBy
90, 98, 110, 120, 135

Choose the right LINQ operation

The method after the dot tells LINQ what to do with each item.

Where()

Keeps only items that satisfy a condition.

Example: r => r > 100

Select()

Projects each item into a new value.

Example: r => r * 2

OrderBy()

Sorts items by the selected key.

Example: r => r

Remember: add using System.Linq; before using LINQ extension methods in a console program.

Practice assignment

Apply methods and collections to analyse a batch of meter readings.

YOUR TASK

Meter Reading Analyzer

Store a batch of meter readings in a List<T> and write methods to report the minimum, maximum and average reading.

Use a List<T> for storage and separate methods for each calculation.